YAMLess Case Study — ngpos-wops-databricks

YAMLess: Generating Azure Infrastructure from Code

Case Study — krogertechnology/ngpos-wops-databricks

What is YAMLess?

YAMLess is an **Infrastructure from Code (IFC)** tool. Given an **application** code repository, it produces a complete, policy-validated Azure Terraform workspace targeting **containerized compute** — Azure Container Apps or AKS. It is designed for microservices and background workers that run in Docker containers.

Applying YAMLess to ngpos-wops-databricks

ngpos-wops-databricks is the NGPOS Warehouse Operations Databricks platform. It implements multiple POS operational event pipelines using **Apache Spark / Databricks**:

- **File consumption** — Delta Live Tables (DLT) ingesting files from ADLS Gen2 into Bronze/Silver layers
- **DESP consumers** — Streaming pipelines consuming from Azure Event Hubs (Kafka-compatible) for diesel and additech events
- **Quality jobs** — Fuel consumer QA checks and reporting
- **API consumption** — Location and store locator data ingestion

The repo includes: - `databricks/source/` — PySpark pipeline code (Delta Live Tables, batch jobs, quality tasks) - `databricks/requirements.txt` — Azure SDK, Kafka, Event Hubs, ADLS Gen2, Databricks SDK - `infrastructure/main.tf` — Terraform provisioning a Databricks workspace via Kroger's `diem-databricks-environment` module - `diem/` — Kroger's Databricks deployment framework (init scripts, job templates, cluster config) - `config/` — Cluster and SQL warehouse configuration JSONs

Command run

```
yamless analyze /tmp/ngpos-wops-databricks --no-llm
```

Profile detected

Field	Detected value	Confidence	Notes
Runtime	`python`	medium	`pyproject.toml` at root — nc Dockerfile found
Runtime version	`null`	—	Dockerfile is at `diem/build/Dockerfile`, no root
Shape	`unknown`	low	No HTTP framework, no work packages at root
Database	`null`	—	`databricks/requirements.` not scanned (non-standard path
Storage	`false`	—	`azure-storage-file-datal` not scanned (same reason)
Queue	`false`	—	`azure-eventhub`, `confluent kafka` not scanned (same reasc
Secrets	`DATABRICKS_TOKEN`, `PYTEST_DATABRICKS_TOKEN`	high	Found in test/workflow source files

No Terraform was generated. Even if yamless had detected all dependencies correctly, generating AKS or Container Apps Terraform would be wrong — this workload runs on Databricks (Apache Spark), not in Docker containers.

What the existing infrastructure actually provisions

The `infrastructure/` directory already contains hand-authored Terraform using Kroger's internal `diem-databricks-environment` Terraform Cloud module, which provisions:

- Databricks workspace (Premium SKU)
- Databricks cluster with Kroger's DIEM framework and Datadog/Dynatrace observability agents
- Key Vault for secrets
- ADLS Gen2 integration for Delta Lake storage
- Artifactory integration for private Python package registry (`diem-framework`, `confluent-kafka`, etc.)

This is exactly the correct infrastructure for a Databricks platform — and it was already written by the team.

What this test case reveals about YAMLess

1. No detection of Databricks / analytics compute paradigms

What happened: YAMLess identified `pyproject.toml` at the root and returned `runtime: python`. It then found `DATABRICKS_TOKEN` secrets, shape `unknown`, and no dependencies — and would have generated a near-empty Container Apps workspace if asked to render.

What should happen: YAMLess should detect Databricks repos early and exit with a clear message:

```
✗ This repository appears to target Databricks / Apache Spark compute.
  YAMLess generates Azure Container Apps and AKS infrastructure for
containerized services.
  Databricks workloads require Databricks workspace infrastructure,
which is out of scope.
  The repo's existing infrastructure/ directory already contains the
appropriate Terraform.
```

Detection heuristics (any of these signal a Databricks platform repo): - `databricks-sdk` in any `requirements*.txt` or `pyproject.toml` - `databricks/` directory at root containing source code - `*.dlt.py` files (Delta Live Tables) - `DATABRICKS_TOKEN` or `DATABRICKS_HOST` env var references - `infrastructure/main.tf` referencing a Databricks workspace resource

---

2. Requirements scanning doesn't handle non-standard paths

What happened: All 16 packages in `databricks/requirements.txt` — including `azure-eventhub`, `confluent-kafka`, `azure-storage-file-datalake`, `azure-servicebus`, `azure-keyvault-secrets` — were completely invisible to yamless. The dependency scanner only looks at `requirements.txt`, `requirements-prod.txt`, and `requirements/base.txt` relative to the repo root. `databricks/requirements.txt` doesn't match any of these.

Impact on other repos: This affects any Python repo that organises requirements by subdirectory (common in monorepos and platform repos). If the Databricks paradigm were supported, none of the rich dependency signals (Kafka, Event Hubs, ADLS, Service Bus) would have been surfaced.

Proposed fix (backlog #13): When the root-level requirements files yield no packages, fall back to searching one level deep for any `requirements*.txt` file and scanning those. This would correctly handle `databricks/requirements.txt`, `api/requirements.txt`, `worker/requirements.txt` patterns common in multi-component Python repos.

---

3. Dockerfile not found — `diem/build/Dockerfile` outside search paths

What happened: `dockerfile: false` was returned even though `diem/build/Dockerfile` exists. The Dockerfile scanner only looks at the repo root (`Dockerfile`, `Dockerfile.prod`, etc.).

This is intentional design — the Databricks image is a cluster init image, not the application's own container. But it illustrates that deep Dockerfiles need to be either found or explicitly excluded to avoid confusing analysis results.

What the `databricks/requirements.txt` signals (if it had been read)

For reference, here is what yamless would have detected from `databricks/requirements.txt`:

Package	Signal
`azure-eventhub==5.11.7`	Queue (Event Hubs / Kafka-compatible)
`confluent-kafka==2.15.0`	Queue (Kafka)
`kafka-python==2.2.20`	Queue (Kafka)
`azure-servicebus==7.12.3`	Queue (Service Bus)
`azure-storage-file-datalake==12.18.0`	Storage (ADLS Gen2)
`azure-storage-queue==12.12.0`	Queue
`azure-keyvault-secrets==4.8.0`	(Key Vault — infra, not app dep signal)
`databricks-sdk==0.46.0`	**Databricks platform signal**
`diem-framework==2.26.1`	Kroger internal Databricks framework

Even with correct dependency detection, `databricks-sdk` would be the decisive signal to reject this repo as a non-containerized workload.

Improvement Roadmap from this test

Priority	Change	Backlog
Medium	Detect Databricks/Spark repos and emit clear unsupported-platform message	#13 (new)
Medium	Fall back to subdirectory `requirements*.txt`	new

Priority	Change	Backlog
	scanning when root has none	
Medium	Consolidate infra-only + wrong-platform detection into a single pre-flight guard	extends #12

Summary

`ngpos-wops-databricks` is an Apache Spark / Databricks data engineering platform. It is not a containerized microservice and is outside the scope of what YAMLess generates. The existing `infrastructure/main.tf` already correctly provisions a Databricks workspace using Kroger's internal DIEM Terraform module.

No Terraform was generated — generating Container Apps or AKS infrastructure for a Databricks workload would be architecturally incorrect.

Two new gaps surfaced: 1. **No Databricks/analytics platform detection** — yamless should recognise `databricks-sdk`, `DATABRICKS_TOKEN`, and similar signals and exit with a clear message rather than returning a confusing partial profile. 2. **Non-standard requirements.txt paths not scanned** — `databricks/requirements.txt` was invisible; a subdirectory fallback scan would fix this for any multi-component Python repo.

Generated by YAMLess — https://github.com/zafar-ahmed-kr/YAMLess